

AI-Based Legal Document Analysis And Generation

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

AKSHAYA NAIR S S 231501011

ASWIN K 231501027

NAVITHA J V 231501106

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report “AI-Based Legal Document Analysis and Generation” is the bonafide work of “AKSHAYA NAIR S S(231501011), ASWIN K(231501027), NAVITHA J V(231501106)” VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

No.	Chapter / Section	Page
1	Introduction	1
	1.1 Background and Motivation	
	1.2 Need for Agentic AI	
	1.3 Project Objectives	
2	Problem Statement and Business Case	3
	2.1 Problem Definition	
	2.2 Business and Real-World Context	
3	User and Stakeholder Analysis	5
	3.1 Target Users	
	3.2 Stakeholder Identification	
	3.3 User Requirements	
4	Proposed Agentic AI Solution	7
	4.1 Overall Solution	
	4.2 Key Features	
	4.3 Agent-Based Approach	
5	Dataset / Knowledge Sources	10
6	Tools and Technologies Used	11
7	Agent Architecture	11
	7.1 Agent Components	
	7.2 Tools	
	7.3 Memory	
	7.4 Reasoning Flow	
	7.5 Agent Interactions	
8	Agent Workflow and Orchestration	15
	8.1 Workflow Design	
	8.2 Task Decomposition	

No.	Chapter / Section	Page
	8.3 Agent Coordination	
	8.4 Orchestration Implementation	
9	Agentic RAG Implementation	18
	9.1 Retrieval Architecture	
	9.2 Knowledge Retrieval	
	9.3 Generation Process	
	9.4 Retrieval Quality Evaluation	
10	System Implementation	20
	10.1 Functional Prototype	
	10.2 Source-Code Repository	
11	Experimental Evaluation and Results	22
	11.1 Evaluation Metrics	
	11.2 Testing Methodology	
	11.3 Performance Results	
	11.4 Success-Metric Achievement	
12	Comparison, Innovation and Scalability	24
	12.1 Comparison with Baseline	
	12.2 Advantages of the Proposed System	
	12.3 Innovation	
	12.4 Scalability	
	12.5 Real-World Applicability	
13	Conclusion and Future Work	26
	13.1 Conclusion	
	13.2 Future Work	

1. INTRODUCTION

1.1 Background and Motivation

The legal domain is one of the areas where information plays a very important role in decision-making. Legal professionals regularly work with documents such as case petitions, complaints, agreements, affidavits, judgments, legal notices, contracts, property documents, and other records. These documents can contain a large amount of information distributed across several pages. Important details such as the names of parties, dates, events, claims, legal provisions, evidence, and arguments may appear at different locations within the same document. Understanding the complete case therefore requires careful reading, legal knowledge, and considerable time.

The increasing availability of Artificial Intelligence has created opportunities to automate several information-processing tasks in the legal domain. Natural Language Processing can be used to process human language, while machine learning and deep learning techniques can identify patterns and relationships within textual information. More recently, Large Language Models have demonstrated the ability to understand questions and generate natural-language responses. However, a general-purpose language model may not contain the specific information required to answer questions about a particular legal case. It may also produce information that is not directly supported by the documents provided by the user. This creates a need for systems that combine language generation with reliable retrieval of relevant document information.

The proposed project, AI-Based Legal Document Analysis and Generation, is developed to address this challenge. The system provides an AI-powered environment where users can upload legal documents and interact with the information contained within them. The application processes the uploaded documents, extracts their textual content, divides the content into manageable sections, converts the sections into semantic representations, and stores them in a vector index. When a user asks a question, the system searches for the most relevant information and provides it to a Large Language Model before generating the final response.

The project uses Retrieval-Augmented Generation as one of its core approaches. RAG allows the system to retrieve relevant information from the uploaded document before asking the language model to generate an answer. This makes the generated response more closely connected to the user's case information. The project also incorporates Context-Aware Generation, which allows previous interactions to be considered when answering follow-up questions. These capabilities together create a more useful legal assistance system compared with a simple chatbot that generates responses without document-specific context.

Another important motivation for the project is accessibility. Legal documents often contain technical terminology that can be difficult for ordinary users to understand. An AI-based system can provide a more conversational interface through which users can ask questions in natural language. Instead of manually searching through a lengthy document, a user can ask a question and receive an explanation based on the relevant sections of the document. The system can therefore support preliminary legal research and document understanding while still requiring professional legal verification for actual legal decisions.

1.2 Need for Agentic AI

Conventional software generally performs a predefined operation after receiving an input. Legal document analysis is more complicated because it involves several interconnected tasks. A document must first be uploaded and identified, after which the text must be extracted and cleaned. The extracted content needs to be divided into meaningful chunks and converted into embeddings. These embeddings must then be stored in a searchable vector database. When a user asks a question, the system needs to interpret the query, retrieve relevant information, consider previous conversation context, and finally generate an appropriate response.

This multi-stage nature makes Agentic AI useful for the proposed application. An agentic system can coordinate different components and tools to complete a larger task. In the proposed project, document processing, semantic retrieval, memory management, legal analysis, and language generation are treated as interconnected stages rather than isolated operations. Each component performs a specific responsibility and contributes its output to the next stage.

The agentic approach also makes the system easier to extend. For example, a future version can introduce a dedicated timeline agent that identifies important dates and events, a legal research agent that searches verified legal sources, and a citation verification agent that checks whether the legal provisions mentioned by the system are appropriate. The current architecture provides a foundation for such extensions.

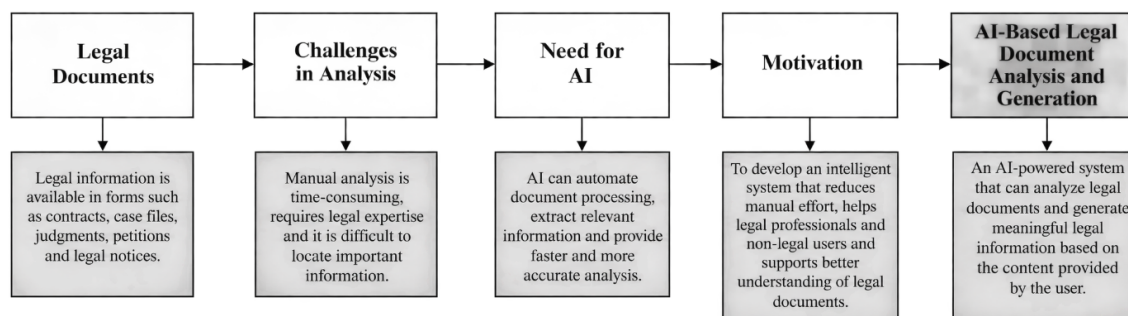


Figure 1. Conceptual Flow of AI-Based Legal Document Analysis

1.3 Project Objectives

The main objective of the project is to develop an intelligent system capable of analyzing legal documents and generating useful information through natural-language interaction. The system is intended to reduce the effort required to manually examine lengthy legal documents by providing semantic search and AI-assisted analysis.

The project aims to provide support for common document formats including PDF, DOCX, and TXT. It automatically extracts textual content from these documents and prepares the content for semantic retrieval. Another objective is to use sentence-transformer embeddings to represent document content in a numerical form and use FAISS for efficient similarity-based retrieval.

The project also aims to integrate Retrieval-Augmented Generation so that the language model can use relevant information from the uploaded document when producing an answer. Context-Aware Generation is used to maintain conversational continuity and support follow-up questions. In addition, the project provides structured legal case analysis covering important aspects such as case facts, legal issues, applicable laws, potential violations, legal strategies, defense arguments, risk assessment, and recommended next steps.

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 Problem Definition

Legal documents are generally lengthy and contain information that may be difficult for non-specialists to interpret. A person attempting to understand a legal case may have to manually identify the important facts, understand the sequence of events, determine the legal issues involved, and search for applicable legal provisions. When several documents are involved, this process becomes even more difficult.

Traditional keyword-based search systems do not completely solve this problem because legal language is highly variable. Two sentences can describe the same legal concept using completely different words. A search system that depends only on exact keyword matching may therefore fail to retrieve important information. For example, a document may refer to unauthorized possession while the user may ask about illegal occupation. Although the concepts may be related, an exact keyword search may not identify the relevant passage.

The proposed system addresses this limitation by using semantic embeddings. Instead of representing a document only as a collection of keywords, the system converts textual information into numerical vectors representing its semantic meaning. User queries are represented using the same approach, allowing the system to identify document sections that are

semantically related to the question.

The second major problem addressed by the project is the limitation of standalone language models. A language model may generate fluent responses but may not have access to the user's specific case information. The proposed RAG architecture solves this by retrieving relevant document information and adding it to the prompt before generation. This provides the model with case-specific context during response generation.

2.2 Business and Real-World Context

The proposed system can provide value in several real-world legal environments. Advocates often spend significant time reviewing documents before preparing arguments or advising clients. A system capable of quickly locating relevant sections and summarizing case information can assist with preliminary document review.

Legal research is another area where the system can provide support. Researchers may need to examine long documents to locate particular facts or legal issues. Semantic retrieval can reduce the effort involved in finding relevant information. Similarly, law students can use the system as an educational tool to understand how facts, legal issues, and applicable laws are related. The system can also benefit individuals who receive legal documents but do not have extensive knowledge of legal terminology. A conversational interface allows users to ask questions in natural language instead of navigating complex document structures manually. The system can explain the content in a more accessible manner.

However, the system is designed as an AI-assisted legal information and analysis tool rather than a replacement for a qualified advocate. AI-generated information may contain errors and therefore should be reviewed by an appropriate legal professional before being used for legal action.

3. USER AND STAKEHOLDER ANALYSIS

3.1 Target Users

The primary target users of the system are advocates, legal researchers, law students, paralegals, legal support staff, and individuals who need to understand legal documents. Advocates can use the system during preliminary case analysis to locate relevant information within lengthy documents. Researchers can use semantic search to explore case material more efficiently. Students can use the application to understand legal documents and practice case analysis.

Individuals who are unfamiliar with legal terminology can also benefit from the conversational nature of the system. Instead of searching through a document manually, they can ask direct questions about the content. The generated response can help them understand the general information contained in their document.

3.2 Stakeholder Identification

The system has several stakeholder groups with different requirements. Legal professionals are concerned with the relevance and reliability of the information retrieved from documents. Clients are primarily interested in understanding the information associated with their cases. Students and researchers require an accessible environment for learning and document exploration. Developers and administrators are responsible for maintaining the software infrastructure and ensuring that the different AI components operate correctly.

The system therefore needs to balance usability, technical performance, privacy, reliability, and extensibility. Since legal documents can contain sensitive information, privacy and secure handling of uploaded documents are particularly important considerations.

3.3 User Requirements

The proposed system is designed to provide users with a simple and convenient platform for interacting with legal documents using Artificial Intelligence. The primary user requirement is the ability to upload legal documents in commonly used formats such as PDF, DOCX, and TXT. The system should accept the uploaded document, extract its textual content, and process the information automatically without requiring the user to manually convert or prepare the document.

The user should be able to enter questions or case-related queries using natural language. The system should understand the query and identify the most relevant information from the uploaded legal document. Instead of requiring the user to search for exact keywords, the system should provide semantic retrieval so that conceptually related information can also be identified. This

makes the interaction easier for users who may not know the exact terminology used in the document.

Another important requirement is that the system should provide meaningful and understandable responses based on the available case information. The generated response should be relevant to the user's query and should use the information retrieved from the uploaded document. The system should support different types of legal queries, including questions related to case facts, legal issues, applicable laws, potential violations, strategies, risks, and other relevant case information.

The system should also maintain the context of the conversation. Users may ask several questions about the same case without repeating the complete case information every time. Therefore, the system should remember relevant previous interactions and use them when responding to follow-up questions. This requirement is handled through the conversation-memory component of the proposed architecture.

The system should provide structured legal analysis when the user requests a case analysis. The analysis should organize important information into understandable sections such as case summary, key facts, legal issues, relevant laws, potential violations, legal strategies, defense arguments, risk assessment, and recommended next steps. This structured output helps users understand the different aspects of a legal case without having to manually organize the information themselves.

The system should also provide a mechanism for starting a new session. When a user finishes working with one case and wants to analyze another case, the previous conversation and associated document information should be cleared. This requirement helps prevent information from one case from being incorrectly used while analyzing another case.

From a usability perspective, the system should provide a simple web-based interface that allows users to upload documents, enter questions, view responses, and perform case analysis without requiring technical knowledge. The interface should provide clear interaction between the user and the AI system and should present generated information in an understandable format.

The system should also provide reasonable reliability during document processing and AI interaction. If an unsupported document is provided or an error occurs during processing, the system should provide an appropriate response instead of producing misleading information. The application should also provide system-status and health information so that the availability of important backend services can be monitored.

Since legal documents may contain sensitive information, privacy is also an important user requirement. The proposed system uses a locally hosted Llama 3 model through Ollama,

providing a local-first approach for language generation. The system should be further enhanced in future versions with stronger authentication, access control, encryption, and secure document management to support production-level legal applications.

4. PROPOSED AGENTIC AI SOLUTION

4.1 Overall Solution

The proposed system combines document processing, semantic retrieval, contextual memory, and language generation into a unified legal AI platform. The user begins by uploading a document or providing a case description. The backend identifies the input type and extracts the available textual information.

The extracted text is cleaned and divided into smaller chunks. Each chunk is converted into an embedding using the Sentence Transformers model. These embeddings are stored in a FAISS index. The index acts as a searchable representation of the uploaded case.

When the user asks a question, the system converts the question into an embedding and searches the FAISS index. The most relevant chunks are retrieved based on semantic similarity. These retrieved chunks are combined with the current question and relevant conversation history. The combined context is then passed to the Llama 3 language model through Ollama.

The language model generates the final response, which is returned through the web application. This architecture creates a complete pipeline from document upload to AI-assisted legal response.

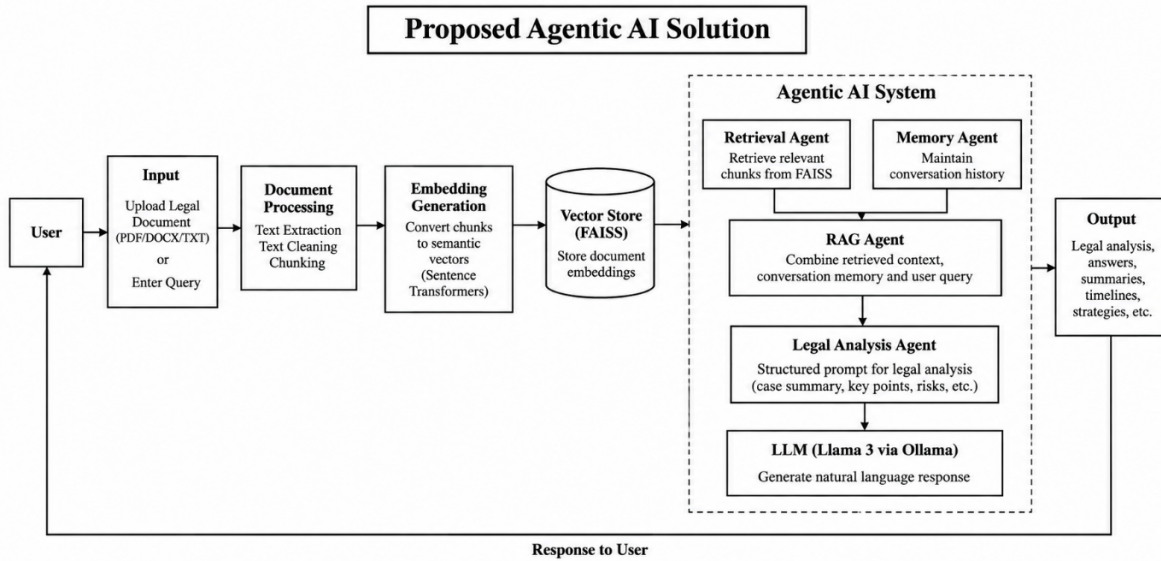


Figure 2. Proposed Solution

4.2 Key Features

The application provides document upload, semantic search, conversational interaction, structured case analysis, contextual memory, and local language-model execution. It supports PDF, DOCX, and TXT documents and processes the uploaded content automatically. The structured analysis functionality is designed to provide information about important case facts, legal issues, relevant laws, potential violations, legal strategies, defense arguments, risk levels, and recommended next steps. The conversational interface allows users to ask questions about the case and receive responses based on the retrieved document context.

The application also includes session-management functionality. Users can reset the current session, which clears the conversation memory and associated document index. System health and status endpoints are provided to check whether the required backend services are operating correctly.

The proposed system is designed to simplify the analysis of legal documents by combining document processing, semantic retrieval, conversational AI, and structured legal analysis.

- Supports uploading legal documents in PDF, DOCX, and TXT formats.
- Automatically extracts and preprocesses the textual content from uploaded documents.
- Converts document content into semantic embeddings for meaningful information retrieval.
- Uses Retrieval-Augmented Generation to provide relevant document context to the language model.

- Maintains conversation history to understand follow-up questions related to the same case.
- Provides AI-based analysis of legal cases using the information available in the document.
- Generates structured information related to case facts, legal issues, applicable laws, risks, strategies, and defense arguments.
- Allows users to interact with the system using natural-language queries.
- Uses Llama 3 through Ollama for AI-based response generation.
- Provides session-reset functionality for starting analysis of a new case.
- Provides system health and status information for monitoring the application.

4.3 Agent-Based Approach

The agentic nature of the system comes from the coordination between several intelligent components. The document processor handles the conversion of unstructured files into text. The embedding component creates semantic representations of the content. The vector retrieval component identifies relevant information. The memory component maintains the conversation state, while the language model produces the final response. This approach separates responsibilities across different modules and allows the system to perform complex legal-analysis tasks through a coordinated workflow. The architecture can later be expanded by adding specialized agents for additional legal tasks.

The proposed system follows an agent-based approach in which different AI components work together to perform the overall task of legal document analysis. Instead of depending on a single component to process the document, retrieve information, analyze the case, and generate the response, the system divides these responsibilities among different specialized components. This allows each component to focus on a specific task while contributing to the final result. The process begins when the user uploads a legal document or enters a legal query. The document processing component extracts and prepares the content, after which the information is converted into semantic embeddings and stored in the FAISS vector store. When the user submits a query, the retrieval component searches the stored document information and identifies the relevant sections based on semantic similarity.

The retrieved information is then combined with the user's query and the available conversation context. The RAG component prepares this information for the language model, while the memory component helps maintain the context of previous interactions. The legal analysis component organizes the available information for case analysis, and the Llama 3 model running

through Ollama generates the final natural-language response.

This agent-based approach enables the different components to interact and coordinate their tasks rather than operating independently. It provides a flexible architecture in which additional specialized agents can be introduced in the future for tasks such as legal research, timeline generation, citation verification, risk assessment, and document comparison. Thus, the agent-based approach provides the foundation for developing an intelligent and extensible legal document analysis system.

5. DATASET / KNOWLEDGE SOURCES

The proposed system does not depend on a single fixed machine-learning dataset for its primary operation. Instead, it uses legal documents supplied by the user as the primary knowledge source. These documents may contain case details, legal arguments, facts, dates, claims, and other information relevant to the user's query. The system supports PDF, DOCX, and TXT documents. PDF content is extracted using appropriate PDF-processing libraries, while DOCX files are processed using the python-docx library. Plain-text files are read directly. After extraction, the content undergoes preprocessing before being divided into chunks.

The uploaded document is converted into a searchable knowledge representation using semantic embeddings. Each chunk is represented as a vector, and the collection of vectors forms the knowledge base for the current case. This approach means that the system can dynamically adapt to different cases instead of depending on a single predefined document collection.

The legal-analysis component is designed around the Indian legal context and includes awareness of major areas of Indian law such as the Bharatiya Nyaya Sanhita, Indian Penal Code where relevant, Transfer of Property Act, Registration Act, Specific Relief Act, Limitation Act, and land and revenue-related laws. These legal references provide additional context for the analysis, while the uploaded case remains the main source for case-specific information.

6. TOOLS AND TECHNOLOGIES USED

The backend of the application is developed using Python and FastAPI. Python provides the main programming environment for document processing, vector operations, AI integration, and application logic. FastAPI is used to create the REST API through which the frontend communicates with the backend. Uvicorn acts as the application server for running the FastAPI application. The project uses Sentence Transformers for semantic embedding generation. The all-MiniLM-L6-v2 model is used to convert text into 384-dimensional vectors. These vectors are stored and searched using FAISS, which provides efficient similarity-based retrieval.

The language-generation component uses Llama 3 through Ollama. Ollama provides a mechanism for running the language model locally, allowing the system to communicate with the model through a local service. This local-first design can be useful when working with sensitive legal documents.

The document-processing layer uses pdfplumber and PyPDF2 for PDF extraction and python-docx for DOCX processing. NumPy supports numerical operations, while Pydantic provides data validation. HTTPX is used for communication between the backend and the Ollama service. The frontend uses HTML, JavaScript, and Tailwind CSS to provide an interactive web interface.

7. AGENT ARCHITECTURE

7.1 Agent Components

The architecture is divided into several functional components. The document extraction component receives the uploaded document and converts its contents into text. The text-processing component cleans the extracted information and prepares it for chunking. The chunking component divides the document into smaller sections so that individual portions can be retrieved efficiently.

The embedding component converts each chunk into a semantic vector using the Sentence Transformers model. The vector-store component stores these vectors in FAISS and performs similarity searches when the user submits a query. The RAG component connects the retrieval system with the language model. It retrieves relevant document sections and constructs the context supplied to the model. The memory component maintains the previous conversation history, while the legal analysis component provides structured prompts for case analysis. The LLM component communicates with Llama 3 through Ollama and generates the final natural-language response.

The different components are designed to work together as an integrated intelligent pipeline rather than as independent modules. Each component performs a specific responsibility and passes its output to the next stage. This separation of responsibilities is important because legal document processing involves multiple operations that cannot be efficiently performed by a single component. For example, the language model should not be responsible for extracting text from a PDF or searching through thousands of document vectors. These operations are handled by dedicated components before the information reaches the language model. The architecture also follows a modular design principle. The document-processing components can be improved without changing the language-generation component, while the embedding model can be replaced without redesigning the frontend. Similarly, the vector database can be upgraded in the future if the number of documents increases. This modularity makes the architecture flexible and easier to maintain.

Another important aspect of the architecture is the separation between the user's original information and the generated response. The uploaded document is first converted into a searchable representation, while the LLM receives selected relevant information rather than the entire document every time. This reduces unnecessary processing and allows the system to focus on the portions of the document that are most relevant to the user's current question. The architecture also supports the concept of specialized intelligence. Document processing is handled by document-processing modules, retrieval is handled by the vector-search component, conversation context is handled by the memory component, and natural-language generation is handled by the LLM. These specialized responsibilities collectively create an agentic workflow in which multiple components cooperate to accomplish the overall task of legal document analysis.

7.2 Tools

The system uses several specialized tools as part of its architecture. Document parsers handle different file formats, the embedding model converts text into vectors, FAISS performs semantic retrieval, the memory module maintains conversation state, and Ollama provides access to the local language model. FastAPI coordinates these components through API endpoints. The document-processing tools are responsible for transforming unstructured files into information that can be processed by the AI pipeline. PDF files require specialized extraction mechanisms because their content may be distributed across pages and may contain different formatting structures. DOCX files are processed using document-specific libraries, while plain-text files can be processed more directly. By supporting multiple formats, the system becomes more practical for real-world legal document analysis.

The embedding model is another important tool because it creates the semantic representation required for intelligent retrieval. Instead of treating the document as a collection of individual keywords, the embedding model converts text into numerical vectors. These vectors allow the system to compare the meaning of a user's query with the meaning of different document sections.

Ollama is used as the local interface for the Large Language Model. Through Ollama, the backend can communicate with the Llama 3 model and provide it with the retrieved context and user query. The use of a locally running model also provides an architecture that can be operated without depending entirely on external cloud-based AI APIs.

FastAPI acts as the communication and orchestration layer between these tools. It provides the API endpoints required for document upload, case analysis, chat interaction, session reset, and system monitoring. Therefore, FastAPI plays an important role in connecting the frontend with the different AI components.

7.3 Memory

Conversation memory is important because legal questions are often dependent on previous information. The system therefore maintains a conversation history that can be used when processing follow-up questions. The memory component uses a controlled context window so that the conversation does not grow indefinitely and consume excessive model context.

7.4 Reasoning Flow

Conversation memory is important because legal questions are often dependent on previous information. The system therefore maintains a conversation history that can be used when processing follow-up questions. The memory component uses a controlled context window so that the conversation does not grow indefinitely and consume excessive model context. Without memory, every question would be treated as an independent request. This would require the user to repeatedly provide the same case information whenever they asked a follow-up question. Such an interaction would make the system less natural and less useful. With memory, the system can maintain information about previous questions and responses and use that information when interpreting subsequent queries.

The reasoning process starts with the user's query. The query is converted into an embedding and compared against the vectors stored in FAISS. The system retrieves relevant document chunks and combines them with the user's question and previous conversation context. This information is then included in the prompt sent to the language model.

The generated response is returned to the user. This process allows retrieval and generation to work together rather than allowing the language model to operate without access to the case document.

7.5 Agent Interactions

The components interact sequentially through the backend. The upload system communicates with the extraction and indexing components. The chat system communicates with the retrieval and memory components. The retrieved information is then passed to the LLM component. The response from the LLM is finally returned to the frontend. The interaction between these components creates a coordinated workflow. When a user uploads a document, the frontend sends the file to the FastAPI backend. The backend identifies the file type and invokes the appropriate extraction mechanism. The extracted content is then processed and divided into chunks. The embedding component converts the chunks into vectors, and the vector-store component stores them in FAISS.

Once the document has been indexed, the system is ready to answer questions about its contents. When the user submits a question, the backend receives the query and sends it through the retrieval process. The query embedding is compared with the stored document embeddings, and the most relevant chunks are returned. At the same time, the memory component provides relevant information from the previous conversation. The retrieved document context and conversation context are then combined with the user's current question. This combined information is passed to the LLM through Ollama. The LLM generates the response and sends it back to the backend. The backend then returns the response to the frontend, where it is displayed to the user. In the case of streaming interaction, the response can be delivered progressively rather than waiting for the entire generation process to finish.

The interaction between the agents can therefore be viewed as a continuous information flow. The document-processing components create the knowledge base, the retrieval component identifies relevant knowledge, the memory component provides conversational information, and the LLM transforms the combined context into a natural-language response. This interaction model also makes the system extensible. Additional components can be inserted into the workflow without changing the basic architecture. For example, a future timeline-generation component could extract dates and events from retrieved legal content before presenting them chronologically. Similarly, a citation-verification component could examine legal references generated by the LLM and compare them with a verified legal knowledge source.

8. AGENT WORKFLOW AND ORCHESTRATION

8.1 Workflow Design

The system workflow begins when the user provides a legal document or case description through the application interface. The input can be provided in the form of a supported document such as PDF, DOCX, or TXT, or as directly entered case information. The backend receives the input and determines the appropriate processing method based on the type of information provided. For uploaded documents, the extraction component converts the document contents into machine-readable text. The extracted information is then cleaned and prepared for further processing so that unnecessary formatting and irrelevant characters do not affect the subsequent stages.

After preprocessing, the extracted text is divided into smaller chunks. Chunking allows the system to work with individual portions of a lengthy legal document instead of processing the complete document for every query. Each chunk is then converted into a semantic embedding using the Sentence Transformers model. These embeddings represent the meaning of the corresponding document sections in numerical form. The generated embeddings are inserted into the FAISS vector index, creating a searchable representation of the uploaded legal document.

Once the document has been indexed, the user can interact with the system through the chat interface. When the user submits a query, the system processes the query using the same embedding mechanism used for the document. The query embedding is compared with the vectors stored in FAISS, and the system performs a similarity search to identify the document chunks that are most relevant to the question. This enables the system to retrieve information based on semantic similarity rather than depending only on exact keyword matching.

The retrieved chunks are then used as contextual information for the response-generation process. The system combines the retrieved document context with the user's current question and relevant conversation history maintained by the memory component. This combined information forms the contextual prompt that is provided to the language model. The Llama 3 model, accessed through Ollama, processes the contextual prompt and generates the final response.

The generated response is then returned to the frontend and displayed to the user. Depending on the API operation being used, the response can either be returned as a complete response or delivered progressively through the streaming mechanism. This workflow allows the system to move systematically from document input to processing, retrieval, contextual reasoning, and final response generation.

The overall workflow is designed to ensure that each stage contributes specific information to the next stage. The document-processing stage creates the searchable knowledge base, the retrieval stage identifies relevant information, the memory component provides conversational context, and the language model transforms the combined information into a natural-language response. This coordinated workflow forms the core operational process of the proposed Agentic AI system.

8.2 Task Decomposition

The overall legal document-analysis task is decomposed into several smaller stages so that each operation can be handled independently. This decomposition is necessary because processing a legal document involves different types of operations, including file handling, text processing, semantic representation, information retrieval, context management, and language generation.

The first stage is document ingestion, where the system receives the user's legal document or case description. The extraction stage then converts the available document into text. After extraction, the chunking stage divides the text into smaller sections suitable for semantic processing. The embedding stage converts these sections into numerical representations that can be stored and searched using FAISS. The retrieval stage is responsible for identifying the information that is most relevant to the user's query. Instead of providing the complete document to the language model, the system retrieves selected portions that are related to the question. This reduces unnecessary processing and allows the model to focus on relevant case information.

The memory stage handles conversational context. It allows the system to use relevant information from previous interactions when processing a new question. This is particularly useful when users ask multiple questions about the same case. The final stage is response generation, where the LLM uses the retrieved information, conversation context, and current query to produce the response. This decomposition makes the application easier to debug, test, and extend. Each stage can be examined independently when an error occurs. For example, an extraction problem can be identified separately from a retrieval problem, while an issue with the generated response can be investigated independently from the vector-search process. The modular task decomposition also makes it possible to replace or improve individual components without completely redesigning the system.

8.3 Agent Coordination

The different components of the system are coordinated through the FastAPI backend. The backend functions as the central communication layer between the frontend, document-processing modules, retrieval system, memory component, legal analysis component, and language model. Each API route is responsible for a particular application operation and ensures that the appropriate processing components are invoked.

The upload routes handle document ingestion and initiate the extraction and indexing process. Once the document has been processed and indexed, the chat routes can access the resulting vector representation when the user submits questions. The chat process coordinates query embedding, similarity search, retrieval of relevant document chunks, memory retrieval, context construction, and communication with the LLM. The analysis routes provide structured case analysis by sending the relevant case information to the analysis component and language model. The reset routes are responsible for managing session state by clearing the current conversation and associated document information when a new session is started. Through these coordinated routes, the backend ensures that the different components operate in the required sequence. The coordination between retrieval and memory is particularly important. Retrieval provides information from the actual uploaded document, while memory provides information from the ongoing conversation. These two sources are combined before the final prompt is sent to the language model. This allows the system to consider both the case document and the user's previous interactions when generating a response.

The coordination mechanism also supports future expansion of the system. Additional specialized components can be introduced without changing the complete workflow. For example, a future timeline-generation agent could process the retrieved information to identify important dates and events, while a citation-verification agent could examine legal references before the final response is presented to the user.

8.4 Orchestration Implementation

The backend acts as the main orchestration layer that connects the user interface with the different AI and processing components. When a document is uploaded, the backend controls the sequence from document extraction to preprocessing, chunking, embedding generation, and vector indexing. Once this process is completed, the resulting vector index can be reused for multiple queries from the same case. This approach improves efficiency because the system does not need to process the entire document again every time the user asks a new question. The document is processed and indexed once, after which individual queries can use the existing vector representation. When a new query arrives, only the query needs to be converted into an embedding before the similarity search is performed.

The orchestration layer also controls the flow of contextual information toward the language model. Relevant document chunks retrieved through FAISS are combined with the current question and available conversation memory. The resulting information is passed to the LLM through the Ollama interface. The generated response is then returned through the backend to the frontend. The orchestration design provides a clear separation between the application interface and the underlying AI components. The frontend does not need to directly communicate with FAISS, the embedding model, or the LLM. Instead, all requests pass through the backend, which manages the appropriate sequence of operations. This makes the system easier to maintain and provides better control over how information moves through the application.

The orchestration architecture also supports session management. When a user finishes working with one case, the reset functionality can clear the current vector index and conversation memory. This ensures that information associated with an earlier case does not unnecessarily remain available when processing a new case.

Overall, the workflow and orchestration mechanism provide a structured path from user input to AI-generated output. The system combines document processing, semantic retrieval, memory, and language generation in a coordinated manner, allowing the proposed Agentic AI solution to perform legal document analysis through multiple interconnected stages rather than relying on a single processing step.

9. AGENTIC RAG IMPLEMENTATION

9.1 Retrieval Architecture

Retrieval-Augmented Generation forms one of the most important parts of the proposed system. The purpose of RAG is to provide the language model with information retrieved from the user's document before generating an answer.

The uploaded document is first divided into chunks. Each chunk is passed through the embedding model to generate a numerical representation. These vectors are stored in FAISS. When the user asks a question, the same embedding model converts the query into a vector. FAISS then identifies document vectors that are semantically similar to the query vector.

The retrieved chunks are passed to the generation component as context. This creates a connection between the user's question, the actual case document, and the generated response.

9.2 Knowledge Retrieval

The project uses all-MiniLM-L6-v2 for embedding generation. The model creates 384-dimensional embeddings that represent the semantic characteristics of the input text. The vector database uses similarity search to identify relevant chunks. The configured retrieval process considers the most relevant results and applies a similarity-distance threshold. This allows the system to avoid including document sections that are not sufficiently related to the user's question.

9.3 Generation Process

After retrieval, the system constructs a prompt containing the retrieved information and the user's question. Conversation history can also be included to preserve context across multiple interactions. This prompt is sent to Llama 3 through Ollama. The model generates a natural-language answer using the supplied information. The system instructions encourage the model to use the retrieved context and avoid inventing unsupported facts when sufficient information is not available.

9.4 Retrieval Quality Evaluation

The quality of the retrieval system is important because the language model's response depends on the information retrieved from the document. Retrieval quality can be evaluated using Precision@K, Recall@K, Hit Rate, Mean Reciprocal Rank, and context relevance. The current implementation uses a configured top-K retrieval approach and a distance threshold to control which results are considered relevant.

For the final project evaluation, a labelled test collection can be created in which expected document sections are identified for different legal questions. The retrieved results can then be compared against these expected sections to calculate quantitative retrieval metrics.

10. SYSTEM IMPLEMENTATION

10.1 Functional Prototype

The proposed system has been implemented as a web-based application called Wolfly – Advocate AI. The application provides users with a centralized interface for uploading legal documents and interacting with the AI system. The frontend contains options for advocate chat, case analysis, legal references, case-file upload, and session management. The user can upload a legal document through the interface and wait for the system to process and index its contents. Once processing is completed, the user can ask questions related to the uploaded case.

The application also provides structured analysis capabilities. The system can organize its analysis around case summary, important facts, legal issues, applicable laws, possible violations, legal strategies, defense arguments, risk assessment, and recommended next steps. These features make the system more useful than a simple conversational chatbot because the output is organized around important legal-analysis categories.

Screenshots of the actual working prototype can be inserted in this section. The screenshots should show the home interface, document upload interface, chat interface, case analysis results, legal strategy section, and risk assessment interface.

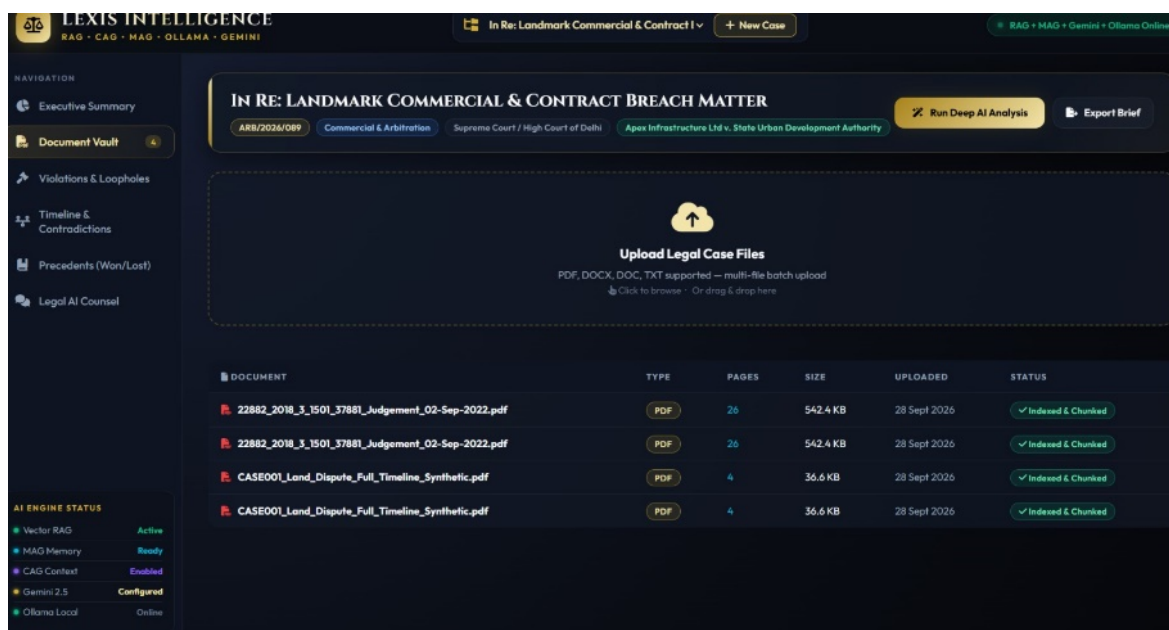


Figure 3. Functional Prototype – Document Vault (Legal Case File Upload)

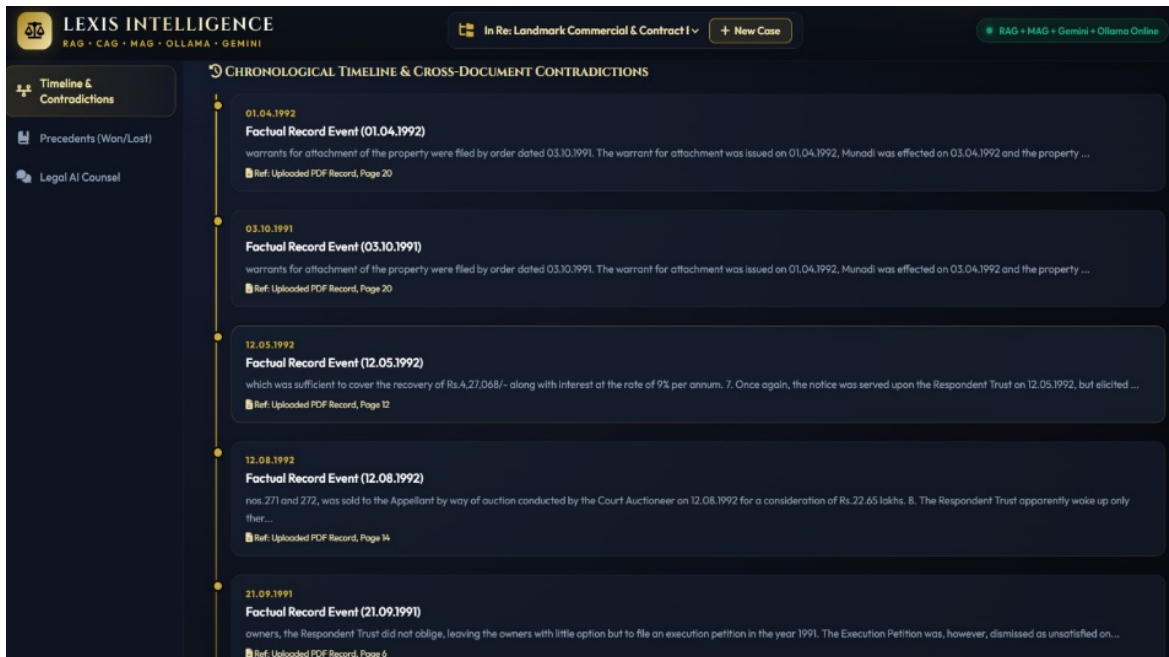


Figure 4. Functional Prototype – Timeline & Contradictions

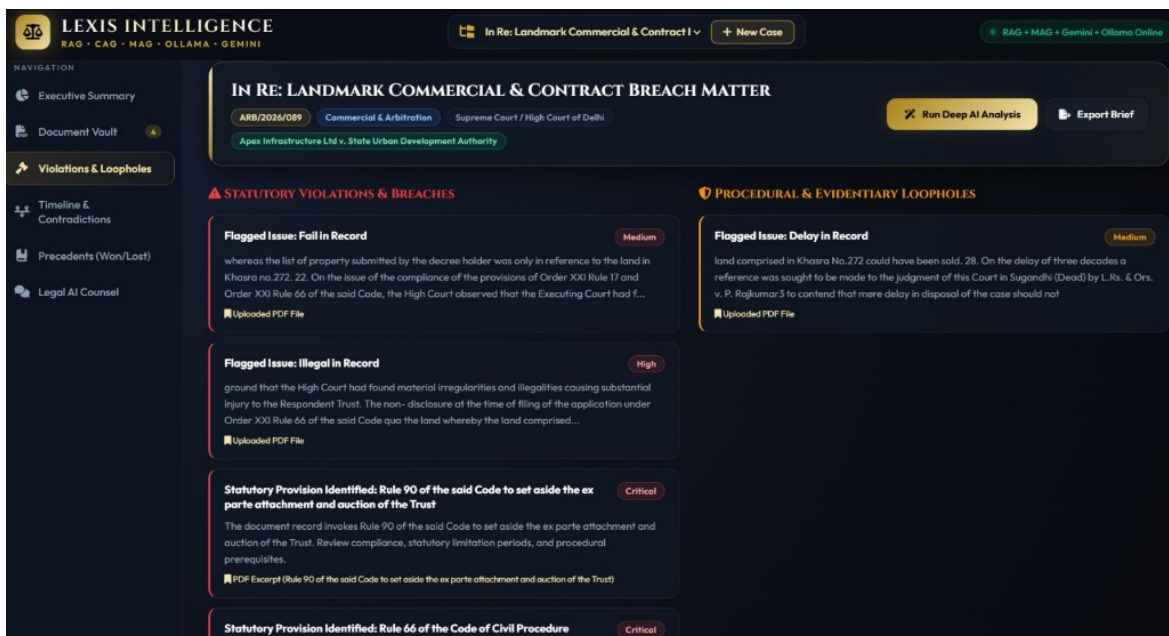


Figure 5. Functional Prototype – Violations & Loopholes

10.2 Source-Code Repository

The source code follows a modular backend architecture. The main backend application is separated from the document-processing pipeline and AI engine. The pipeline contains modules responsible for document extraction, chunking, embedding generation, and vector storage. The engine contains modules for LLM communication, RAG, memory, and legal analysis. Separate router modules handle uploading, analysis, chat, and session reset operations.

This modular structure improves maintainability because changes to one component can be made without significantly affecting the rest of the application. It also makes future development easier because new AI capabilities can be added as separate modules.

11. EXPERIMENTAL EVALUATION AND RESULTS

11.1 Evaluation Metrics

The system can be evaluated using metrics related to document processing, retrieval quality, response generation, and overall system performance. Document-processing evaluation can determine whether PDF, DOCX, and TXT documents are successfully uploaded, extracted, processed, and indexed without major errors. This helps verify the reliability of the initial stages of the system. Retrieval evaluation can determine whether the system returns relevant document sections for a given question. Metrics such as Precision@5 and Recall@5 can be used to measure the quality of the retrieved results. Precision@5 indicates how many of the top five retrieved chunks are relevant to the user's query, while Recall@5 measures how effectively the system retrieves the relevant information available in the document.

Generation quality can be evaluated by examining whether the response is relevant to the user's question, whether it is supported by the retrieved context, and whether structured analysis fields are correctly generated. The generated response can also be checked for consistency with the uploaded legal document. System performance can be evaluated using response time, successful API operations, and resource usage during document processing and query handling.

11.2 Testing Methodology

Testing should be performed using representative legal documents containing different types of information. The test documents can contain multiple parties, legal provisions, dates, factual descriptions, claims, case-related information, and other relevant legal content. Using documents with different structures helps determine whether the system can handle variations in document content. The documents are uploaded to the system and processed through the complete pipeline. The extracted text is cleaned, divided into chunks, converted into embeddings, and stored in the FAISS vector index. After indexing, users can submit different types of questions related to the document. The retrieved results are examined to determine whether the correct portions of the document are identified.

The generated response is then reviewed for relevance, contextual correctness, and consistency with the retrieved information. Different categories of questions can be tested, including questions about case facts, legal issues, applicable laws, risks, strategies, and possible defense arguments. This helps evaluate whether the system can provide useful responses for different legal analysis requirements.

Follow-up questions can also be used to test the conversation-memory functionality. The system should be able to understand references to information discussed in previous turns without requiring the user to repeat the complete case description. This testing verifies whether the context-aware conversation mechanism works correctly across multiple interactions.

11.3 Performance Results

The implemented prototype demonstrates the integration of document processing, semantic retrieval, conversational memory, and LLM-based generation. The system is capable of accepting supported document formats, processing their contents, creating embeddings, storing them in FAISS, retrieving relevant information, and generating responses through the language model. However, numerical performance results should be generated through actual testing rather than estimated values. Metrics such as Precision@5, Recall@5, average response time, successful document-processing rate, and structured-output accuracy can be calculated after conducting a controlled evaluation. Multiple test queries should be used so that the results are not based on a single example.

The evaluation results can be presented using tables and graphs in the final report. For example, retrieval metrics can be presented separately from response-time measurements to clearly show both the quality and efficiency of the system. Any limitations observed during testing, such as incorrect retrieval, incomplete extraction, or longer response time for larger documents, should also be documented as part of the experimental results. The final report can include the measured

values in a results table after testing. This approach ensures that the evaluation section represents actual experimental observations rather than unsupported claims.

11.4 Success-Metric Achievement

The functional objectives of the project are represented by the successful implementation of document uploading, text extraction, document chunking, embedding generation, FAISS indexing, semantic retrieval, conversational memory, structured legal analysis, and LLM-based response generation. Each of these components contributes to the overall objective of developing an AI-based legal document analysis platform.

The system successfully establishes an end-to-end workflow in which a legal document can be provided as input, processed into searchable representations, and used as contextual information for answering user queries. The combination of retrieval and language-model generation allows the system to produce responses based on information available in the uploaded document rather than relying only on general language-model knowledge. The prototype therefore provides the major functional components required for an AI-based legal document analysis platform. Further quantitative evaluation using a larger collection of legal documents and predefined test questions can strengthen the experimental validation of the system. Future testing can also evaluate the system with larger datasets, more complex legal documents, and a wider range of user queries to determine its scalability and robustness.

12. COMPARISON, INNOVATION AND SCALABILITY

12.1 Comparison with Baseline

Traditional legal document search systems commonly rely on keyword-based matching. Such systems require users to know the exact words or phrases that appear in the document. The proposed system uses semantic embeddings, allowing the system to retrieve information based on the meaning of a query.

A conventional chatbot may generate an answer without accessing the user's document. The proposed system instead retrieves relevant document content before generating the answer. This combination of semantic retrieval and language generation provides a more document-aware interaction.

12.2 Advantages of the Proposed System

The proposed system provides several advantages through the integration of different technologies. Semantic retrieval allows users to search documents using natural language. RAG connects the generated response with retrieved case information. Conversation memory provides continuity between multiple questions. Structured case analysis organizes important legal information into understandable categories.

The use of Ollama also enables local execution of the Llama 3 model. This can provide advantages when users prefer not to send sensitive legal documents to external AI services. The modular architecture additionally makes it easier to replace or upgrade individual components.

12.3 Innovation

The main innovation of the project is the combination of semantic document retrieval, conversational context, and AI-based legal analysis within a single platform. The system does not simply store documents or provide a generic chatbot. Instead, it creates a semantic representation of the user's case and dynamically retrieves relevant information when questions are asked.

The integration of RAG and CAG provides both document grounding and conversational continuity. The structured legal-analysis capability further allows the system to organize its output according to important aspects of a legal case.

12.4 Scalability

The system can be extended to support a larger collection of legal documents and multiple cases. The current FAISS implementation provides a foundation for semantic retrieval, while future versions can adopt distributed vector databases for larger deployments.

The backend can also be deployed using cloud infrastructure if required. More advanced embedding models can be incorporated to improve retrieval quality. The architecture can further be extended into a multi-agent system in which different agents specialize in legal research, case analysis, timeline extraction, citation verification, and risk assessment.

12.5 Real-World Applicability

The proposed system can be used for preliminary legal document analysis, case-document summarization, legal research, educational purposes, contract analysis, property-related case analysis, and paralegal assistance. It can help reduce the time required to locate relevant information within lengthy documents and provide users with a more accessible way to interact with legal information.

The system should nevertheless be considered an AI-assisted tool rather than an autonomous legal decision-making system. Legal professionals should verify important legal interpretations, citations, and recommendations before relying on them in actual proceedings.

13. CONCLUSION AND FUTURE WORK

13.1 Conclusion

The AI-Based Legal Document Analysis and Generation project demonstrates the application of modern Artificial Intelligence techniques to the legal document-analysis domain. The system integrates document extraction, text preprocessing, semantic embeddings, vector similarity search, Retrieval-Augmented Generation, Context-Aware Generation, and Large Language Models to create an intelligent legal assistance platform.

The system allows users to upload legal documents and interact with their content using natural-language questions. The document is processed and converted into a searchable semantic representation using Sentence Transformers and FAISS. When a user submits a question, relevant portions of the document are retrieved and provided to the Llama 3 language model through Ollama. This allows the generated response to be based on information associated with the user's case rather than relying only on general model knowledge.

The inclusion of conversational memory allows the system to maintain context across multiple interactions. The structured case-analysis functionality further helps organize information into categories such as case facts, legal issues, applicable laws, potential violations, strategies, defense arguments, risks, and next steps.

The project demonstrates how Agentic AI concepts can be applied to a real-world legal information problem. Instead of using one AI model for every operation, the system coordinates several specialized components to perform document processing, retrieval, contextualization, and generation. This architecture provides a strong foundation for future improvements.

13.2 Future Work

One of the major future enhancements is the implementation of automated legal timeline generation. Such a module can identify dates and important events from a legal document and arrange them chronologically. This would be particularly useful for cases involving multiple events, hearings, notices, transactions, and legal proceedings.

The system can also be enhanced with specialized legal research capabilities. A future legal research agent could retrieve information from verified legal databases and judgments. A

citation-verification component could validate legal sections and reduce the possibility of incorrect references. A dedicated risk-analysis component could provide more systematic assessment of case-related risks.

Additional improvements can include Optical Character Recognition for scanned documents, multilingual support for Indian languages, stronger authentication and authorization, encrypted document storage, and improved legal-domain embedding models. The system can also be evaluated using a larger labelled dataset to measure retrieval accuracy, response quality, factual grounding, and overall system performance.

With these improvements, the proposed system can evolve from a functional prototype into a more comprehensive AI-assisted legal document analysis platform capable of supporting a wider range of legal workflows.